

BDA Assignment-5

Q1) Define Spark DataFrames and explain their significance.

A)

Spark DataFrames

A **DataFrame in Apache Spark** is a **distributed collection of data organized into named columns**, similar to a table in a relational database or a DataFrame in Python's **Pandas** library. It is built on top of **RDDs (Resilient Distributed Datasets)** but provides a **higher-level abstraction** that makes data manipulation easier and more efficient.

In simpler terms, a **Spark DataFrame** is like a **spreadsheet** or a **database table**, but it is distributed across many computers in a cluster and can handle **massive amounts of data** in parallel.

- Each row in a DataFrame represents a record.
- Each column represents a field with a specific data type.
- It supports **structured and semi-structured data** like JSON, CSV, Parquet, Avro, etc.

Example of creating a DataFrame in PySpark:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("DataFrameExample").getOrCreate()
```

```
# Creating a DataFrame from a list of tuples
```

```
data = [("John", 25), ("Sara", 30), ("Mike", 28)]
```

```
columns = ["Name", "Age"]
```

```
df = spark.createDataFrame(data, columns)
```

```
df.show()
```

Output:

```
+----+----+
```

```
|Name|Age|
```

```
+----+----+
```

|John| 25|

|Sara| 30|

|Mike| 28|

+----+----+

Significance of Spark DataFrames

Spark DataFrames are extremely significant in **Big Data analytics** because they combine the **power of distributed computing** with the **simplicity of SQL-like operations**. Here's why they are so important:

1. High-Level Abstraction Over RDDs

- RDDs require you to write complex code for even basic data manipulations.
- DataFrames offer **built-in operations** (like select, filter, groupBy, etc.) that make data processing simpler and more readable.
- Example:
- `df.filter(df.Age > 25).show()`

This is much easier than writing map-reduce logic with RDDs.

2. Optimized Performance with Catalyst Optimizer

- DataFrames are internally optimized by Spark's **Catalyst Optimizer**, which automatically improves the execution plan.
- This leads to **faster query execution** compared to raw RDD transformations.
- You don't need to manually optimize your queries — Spark does it for you.

3. Support for Multiple Data Sources

- DataFrames can read and write data from a variety of sources such as:
 - CSV
 - JSON
 - Parquet
 - ORC

- JDBC (databases)

- This makes them very flexible for integrating with different data systems.

Example:

```
df = spark.read.csv("data.csv", header=True, inferSchema=True)
```

4. Seamless Integration with SQL

- DataFrames can be registered as temporary tables or views and queried using **SQL syntax**.
- This allows data analysts and engineers familiar with SQL to work easily with big data.

Example:

```
df.createOrReplaceTempView("people")
```

```
result = spark.sql("SELECT Name FROM people WHERE Age > 25")
```

```
result.show()
```

5. Better Memory Management and Execution

- DataFrames use **Tungsten Execution Engine**, which optimizes memory usage and CPU efficiency.
- It includes features like **whole-stage code generation** and **off-heap memory management**, leading to faster computation.

6. Interoperability with Machine Learning and Streaming

- DataFrames are a core part of Spark's ecosystem and are used extensively in:
 - **Spark MLlib** for machine learning pipelines.
 - **Spark Structured Streaming** for real-time data streams.
- This makes them a bridge between batch, streaming, and ML workloads.

7. Ease of Use and Less Boilerplate Code

- With built-in functions, you write **fewer lines of code** to achieve the same tasks compared to RDDs.
- This reduces development time and makes code more maintainable.

Q2) Demonstrate essential operations on Spark DataFrames.

A)

Essential Operations on Spark DataFrames

In Apache Spark, **DataFrames** are a powerful abstraction for handling large-scale structured and semi-structured data. Once a DataFrame is created, we can perform many **essential operations** to explore, clean, transform, and analyze the data — all in a way that is **simple, efficient, and similar to SQL**.

Below is a detailed explanation of the most commonly used and essential DataFrame operations, along with examples.

1. Creating a DataFrame

Before performing any operations, we first need a DataFrame. It can be created from various sources like Python lists, CSV files, JSON files, or existing RDDs.

Example – From a list of tuples:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("DataFrameOps").getOrCreate()
```

```
data = [("John", 25), ("Sara", 30), ("Mike", 28)]
```

```
columns = ["Name", "Age"]
```

```
df = spark.createDataFrame(data, columns)
```

```
df.show()
```

Output:

```
+----+----+
|Name|Age|
+----+----+
|John| 25|
|Sara| 30|
|Mike| 28|
+----+----+
```

2. Displaying Data

To quickly inspect the contents of a DataFrame:

- **show()** – Displays the first 20 rows by default.

```
df.show()
```

- **show(n)** – Displays first *n* rows.

```
df.show(2)
```

- **printSchema()** – Displays the structure and data types of the DataFrame.

```
df.printSchema()
```

3. Selecting Columns

Use `select()` to retrieve specific columns from the DataFrame.

```
df.select("Name").show()    # Select one column
```

```
df.select("Name", "Age").show() # Select multiple columns
```

4. Filtering Data (Using `filter()` or `where()`)

To filter rows based on conditions:

```
df.filter(df.Age > 25).show()
```

or

```
df.where(df.Age > 25).show()
```

Output:

```
+----+----+
|Name|Age|
+----+----+
|Sara| 30|
|Mike| 28|
+----+----+
```

5. Adding New Columns (Using `withColumn()`)

We can create a new column or modify an existing one using `withColumn()`.

```
from pyspark.sql.functions import col
```

```
df2 = df.withColumn("AgeAfter5Years", col("Age") + 5)
```

```
df2.show()
```

Output:

```
+---+---+-----+
|Name|Age|AgeAfter5Years|
+---+---+-----+
|John| 25|          30 |
|Sara| 30|          35 |
|Mike| 28|          33 |
+---+---+-----+
```

6. Renaming Columns (Using `withColumnRenamed()`)

To rename a column:

```
df_renamed = df.withColumnRenamed("Age", "CurrentAge")
```

```
df_renamed.show()
```

7. Dropping Columns

Use `drop()` to remove one or more columns:

```
df_dropped = df.drop("Age")
```

```
df_dropped.show()
```

8. Grouping and Aggregation

We can group data by one or more columns and apply aggregation functions like `count()`, `sum()`, `avg()`, etc.

```
from pyspark.sql.functions import avg
```

```
df.groupBy("Age").count().show()    # Count how many have each age
```

```
df.groupBy().agg(avg("Age")).show()  # Average age of all people
```

9. Sorting Data

- **Ascending order:**

```
df.orderBy("Age").show()
```

- **Descending order:**

```
df.orderBy(df.Age.desc()).show()
```

10. Combining DataFrames (Union)

To combine two DataFrames with the same schema:

```
data2 = [("David", 22)]
```

```
df2 = spark.createDataFrame(data2, columns)
```

```
df_union = df.union(df2)
```

```
df_union.show()
```

11. Joining DataFrames

DataFrames can be joined like SQL tables:

```
address_data = [("John", "New York"), ("Sara", "London")]
```

```
columns2 = ["Name", "City"]
```

```
df_address = spark.createDataFrame(address_data, columns2)
```

```
df_joined = df.join(df_address, on="Name", how="inner")
```

```
df_joined.show()
```

Output:

```
+----+---+-----+
```

```
|Name|Age|City  |
```

```
+----+---+-----+
```

```
| John | 25 | New York |
```

```
| Sara | 30 | London |
```

```
+-----+---+-----+
```

12. SQL Queries on DataFrames

DataFrames can be registered as temporary views to run **SQL queries**:

```
df.createOrReplaceTempView("people")
```

```
result = spark.sql("SELECT Name FROM people WHERE Age > 25")
```

```
result.show()
```

13. Descriptive Statistics

We can get summary statistics for numeric columns using `describe()`:

```
df.describe().show()
```

Output:

```
+-----+---+-----+
```

```
| summary | Name |      Age |
```

```
+-----+---+-----+
```

```
| count | 3 |      3 |
```

```
| mean | null | 27.666666666666668 |
```

```
| stddev | null | 2.5166114784235836 |
```

```
| min | John |      25 |
```

```
| max | Sara |      30 |
```

```
+-----+---+-----+
```

Q3) Compare Spark DataFrames with RDDs.**A)**

Feature	RDD (Resilient Distributed Dataset)	DataFrame
Definition	RDD is the low-level, fundamental data structure in Spark, representing a distributed collection of objects processed in parallel.	DataFrame is a higher-level abstraction built on top of RDDs , organized into named columns like a table in a database.
Data Representation	Data is represented as unstructured or semi-structured objects (e.g., tuples, objects).	Data is represented as structured data with schema (columns with names and types).
Abstraction Level	Low-level API – requires manual handling of structure and transformations.	High-level API – provides built-in methods for structured data manipulation.
Ease of Use	More complex to use because you need to define transformations explicitly using functions like map, filter, etc.	Easier and more readable – uses SQL-like expressions and functions (select, filter, groupBy).
Schema Information	No schema – each element is just an object without named fields.	Has a schema – columns are named and have defined data types.
Type Safety	Provides compile-time type safety (in strongly typed languages like Scala and Java).	Provides runtime type safety – schema is checked when operations are executed.
Optimization	No automatic optimization – execution plan is determined by the user's code.	Uses Catalyst Optimizer to automatically optimize queries and execution plans.
Performance	Generally slower due to lack of optimizations and higher overhead in serialization/deserialization.	Faster due to internal optimizations, efficient memory management (Tungsten), and better code generation.

Use of SQL	Cannot use SQL queries directly on RDDs.	Can be queried directly using SQL syntax after registering as a temporary view.
Data Source Support	Requires manual parsing and conversion of data from external sources.	Directly supports various data sources (CSV, JSON, Parquet, ORC, JDBC, etc.).
Interoperability	Not directly compatible with Spark SQL or DataFrame APIs.	Seamlessly integrates with Spark SQL, MLlib, and Structured Streaming.
Code Length	Requires more code to perform even simple operations.	Requires less code – more concise and expressive.
Best Used For	- Low-level transformations and actions. - When fine-grained control over data is needed. - Unstructured data processing.	- Structured or semi-structured data. - SQL-style queries and analytics. - High-level data processing with better performance.